

重庆希微科技有限公司

SWT6XXX 系列芯片 Wi-Fi 移植手册

地址：

上海市浦东新区张江科技园碧波路 889 弄 S1 座 10 层

www.seekwavetech.com

[illegible]

目录

1. 主要思想.....	4
2. 应用层.....	4
2.1 Linux 系统.....	4
2.2 Android 系统.....	8
3 Driver 层.....	11
3.1 Kernel 内部编译.....	11
3.2 Kernel 外部编译.....	11
4 配置和使用.....	12
4.1 log_level.....	12
4.2 Country code.....	12
4.3 skw config.....	13
4.4 MAC 地址.....	13
4.5 固件校准文件.....	13
4.6 查看软件信息.....	14
5 FAQ.....	15

1. 主要思想

WiFi 功能要完整的运行的话， 软件需要有三部分，WIFI 固件，WIFI 驱动，应用层的 wpa_supplicant/hostapd;

固件的加载在驱动里面完成：目前驱动按照功能划分最终生成两个内核模块文件，BSP 部分和 WIFI 部分，BSP 部分根据总线的类型(USB, SDIO, PCIE)分别对应三个 ko 文件 skw_usb_v20.ko、skw_sdio_v20.ko、skw_pcie.ko, wifi 相关的那个是 skw6316.ko;当这两个 KO 加载完成之后，通过 `ifconfig -a` 查看到 wlan0 接口就可以认为底层的初始化工作已经准备好；

应用层的 wpa_supplicant 或者 hostapd 是用来真正的实现 wifi 的业务功能，STA 模式对应 wpa_supplicant，SoftAP 模式对应 hostapd；

2. 应用层

2.1 Linux 系统

2.1.1 wpa_supplicant 配置和编译

➤ 在.config 文件中确认以下选项是开启的

```
CONFIG_DRIVER_NL80211=y
CONFIG_MBO=y
CONFIG_BGSCAN_SIMPLE=y
```

可以根据需要自己开启对应的功能；为了支持漫游功能必须关闭下面选项，

```
#CONFIG_NO_ROAMING=y
```

编译之后会生成 wpa_supplicant, hostapd, wpa_cli, hostapd_cli 四个执行文件；wpa_cli 和 hostapd_cli 可以用来命令行的方式分别跟 wpa_supplicant 和 hostapd 进行动态交互，用来发送命令和查看状态。

2.1.2 wpa_supplicant、hostapd 使用

➤ `wpa_supplicant -Dnl80211 -iwlan0 -c /etc/wpa_supplicant.conf -B`

※:参考配置文件如下，

```
ctrl_interface=/data/vendor/wifi/wpa/sockets
disable_scan_offload=1
update_config=1
p2p_add_cli_chan=1
```

```
p2p_go_ht40=1
p2p_go_vht=1
filter_rssi=-75
pmf=1
sae_pwe=2
external_sim=1
wowlan_triggers=any
bgscan="simple:11:-70:300"
gas_rand_addr_lifetime=0
gas_rand_mac_addr=1
network={
ssid="OPEN-TEST"
scan_ssid=1
key_mgmt=NONE
}
network={
ssid="PSK-TEST"
psk="12345678"
key_mgmt=WPA-PSK FT-PSK WPA-PSK-SHA256
group=CCMP TKIP
}
network={
ssid="WPA3-SAE-TEST"
sae_password="12345678"
proto=RSN
key_mgmt=SAE
pairwise=GCMP-256 CCMP GCMP
group=GCMP-256 CCMP GCMP
ieee80211w=2
}
```

➤ **hostapd -B /etc/hostapd.conf**

参考配置文件如下:

```
interface=wlan0
driver=nl80211
ctrl_interface=/data/vendor/wifi/hostapd/ctrl
ssid=skw_hostapd
country_code=US
hw_mode=a
channel=149
#channel=161
#hw_mode=g //2.4g
#channel=11
```

```
#channel=0
#ht
ieee80211n=1
ht_capab=[HT40+]
#[HT40-][SHORT-GI-20][SHORT-GI-40][DSSS_CCK-40]
#vht
ieee80211ac=1
vht_capab=[SHORT-GI-80][RXLDPC][SU-BEAMFORMEE]
#vht_oper_chwidth=1
#vht_oper_centr_freq_seg0_idx=155
#he
ieee80211ax=1
#he_oper_chwidth=1
#he_oper_centr_freq_seg0_idx=155
#he_oper_centr_freq_seg0_idx=171
#wpa2 conf
#wpa=2
#rsn_pairwise=CCMP
#wpa_passphrase=12345678
#wpa3 conf
#wpa=2
#rsn_pairwise=CCMP
#wpa_key_mgmt=SAE
#ieee80211w=2
#sae_require_mfp=2
#sae_password=12345678
##sae_pwe=2
#wpa2 wpa3
#wpa=2
#wpa_key_mgmt=WPA-PSK SAE
#rsn_pairwise=CCMP
#wpa_passphrase=12345678
#sae_password=12345678
#ieee80211w=1
#sae_require_mfp=1
ignore_broadcast_ssid=0
#ignore_broadcast_ssid=1 //hide ssid
#wps
#eap_server=1
#wps_state=2
#ap_setup_locked=1
#wps_pin_requests=/var/run/hostapd.pin-req
#device_type=6-0050F204-1
#config_methods=push_button
```

```
##device_name=skw AP
##manufacturer=www.seekwave.org
##wps_independent=0
#hostapd_cli -i wlan0 wps_pbc
#hostapd_cli -i wlan0 wps_pin any (pin)
#ieee80211d=1
#wowlan_triggers=any
#supported_rates=10 20 55 110 60 90 120 180 24 360 480 540
#basic_rates=10 20 55 110
#beacon_int=100
#dtim_period=2
#max_num_sta=16
#bridge=
#bssid=fe:fd:fc:12:63:33
#####wpa2 wpa3
#wpa=3
#wpa_key_mgmt=WPA-PSK SAE
#wpa_pairwise=CCMP
#rsn_pairwise=CCMP
#sae_password=12345678
#ieee80211w=1
#sae_require_mfp=1
#####wpa1
#wpa=1
#wpa_passphrase=12345678
#wpa_pairwise=TKIP CCMP
#wpa_key_mgmt=WPA-PSK
```

➤ 注意事项

- 1) ctrl_interface 对应的目录要存在。
- 2) 在启动应用之前确认模块已经成功加载，ifconfig -a 的时候能看到 wlan0 网络设备接口。
- 3) 如果做 softap 的时候，wlan0 被桥接在 br0 下面，需要在配置里面配置 bridge=br0
- 4) softap 主要功能配置列表

序号	功能列表	描述
1	最大连接数	28
2	ACL 功能	支持
3	隐藏 SSID 功能	支持
4	支持带宽	20/40/80 MHz
5	802.11i	WPA/WPA2/WPA3
6	WEP	WEP40/WEP104

2.2 Android 系统

2.2.1 主要思想

Android 系统的 wifi 对比 linux 系统主要是增加了 framework 和上层的处理，把 wpa_supplicant 作为了 wifi_hal 层的一部分，为了适应这个框架，需要做一些相应的工作包括：

- (1) 设置 wpa_supplicant service 的启动配置参数

```
on property:vendor.wifi.direct.interface=p2p0
    setprop wifi.direct.interface p2p0
on property:vendor.wifi.direct.interface=p2p-dev-wlan0
    setprop wifi.direct.interface p2p-dev-wlan0

service wpa_supplicant /vendor/bin/hw/wpa_supplicant \
-Dn180211 -iwlan0 -c/vendor/etc/wifi/wpa_supplicant.conf -B
user wifi
group wifi inet keystore
interface aidl android.hardware.wifi.suppliment.ISuppliment/default
class main socket wpa_wlan0 dgram 660 wifi wifi disabled oneshot
```


p2p 接口设置

```
//./1.6/default/wifi_chip.cpp
std::string getPredefinedP2pInterfaceName() {
    std::array<char, PROPERTY_VALUE_MAX> primaryInterfaceName;
    char p2pParentInterfaceName[100];
    std::string p2pDeviceInterfaceName = "";
    std::array<char, PROPERTY_VALUE_MAX> buffer;
    property_get("wifi.direct.interface", buffer.data(), "p2p0"); //默认 default p2p interface name 是 p2p0
    if (strncmp(buffer.data(), P2P_MGMT_DEVICE_PREFIX, strlen(P2P_MGMT_DEVICE_PREFIX)) == 0) {
        /* Get the p2p parent interface name from p2p device interface name set
        * in property */
        strncpy(p2pParentInterfaceName, buffer.data() + strlen(P2P_MGMT_DEVICE_PREFIX),
                strlen(buffer.data()) - strlen(P2P_MGMT_DEVICE_PREFIX));
        if (property_get(kActiveWlanInterfaceNameProperty, primaryInterfaceName.data(), nullptr) == 0) {
            return buffer.data();
        }
        /* Check if the parent interface derived from p2p device interface name
        * is active */
        if (strncmp(p2pParentInterfaceName, primaryInterfaceName.data(),
                    strlen(buffer.data()) - strlen(P2P_MGMT_DEVICE_PREFIX)) != 0) {
            /*
            * Update the predefined p2p device interface parent interface name
            * with current active wlan interface
            */
            p2pDeviceInterfaceName += P2P_MGMT_DEVICE_PREFIX;
            p2pDeviceInterfaceName += primaryInterfaceName.data();
            LOG(INFO) << "update the p2p device interface name to " << p2pDeviceInterfaceName.c_str();
            return p2pDeviceInterfaceName;
        }
    }
    return buffer.data();
}
```

目前我们的 p2p 支持两种方式，一个是 p2p-dev-wlan0；另一个是 p2p0 的方式，p2p0 的方式里面目前支持可以不创建单独的 group interface（-Puse_p2p_group_interface=1）。驱动的配置里面编译的时候需要增加 CONFIG_SKW6316_LEGACY_P2P=y；

(2) 什么时候加载驱动

这个里面每个方案厂商实现的差别比较大；大部分选在在 init.rc 里面去加载 BSP 的驱动，也有动态的在启动的时候根据 dev_vid 和 dev_pid 来做动态预加载；最后在打开 WIFI 功能的时候去加载 skw6316.ko，关闭的时候卸载 skw6316.ko，也有为了更快的开关，选择不去卸载 skw6316.ko。

(3) 实现 vendor 对应的 wifi_hal 功能

这个对应我们代码里面的 hardware/seekwave/wlan/wifi_hal，需要做如下修改：

- frameworks/opt/net/wifi/libwifi_hal/Android.mk

```
diff --git a/libwifi_hal/Android.mk b/libwifi_hal/Android.mk
index c860b7484..b382e0693 100644
--- a/libwifi_hal/Android.mk
+++ b/libwifi_hal/Android.mk
@@ -123,6 +123,8 @@ endif
     LIB_WIFI_HAL := libwifi-hal-emu
     else ifeq ($(BOARD_WLAN_DEVICE), slsi)
         LIB_WIFI_HAL := libwifi-hal-slsi
+    else ifeq ($(BOARD_WLAN_DEVICE), seekwave)
+        LIB_WIFI_HAL := libwifi-hal-skw
     endif
```

- device/XXX/BoardConfig.mk

增加或者修改如下内容:

```
BOARD_WLAN_DEVICE := seekwave
```

- (4) 为 wpa_supplicant 增加 Android vendor cmd 对应的功能

这个对应我们代码里面的 hardware/seekwave/wlan/wpa_supplicant8_lib
需要做如下修改 device/XXX/BoardConfig.mk

```
BOARD_WPA_SUPPLICANT_PRIVATE_LIB := lib_driver_cmd_skw
BOARD_HOSTAPD_PRIVATE_LIB := lib_driver_cmd_skw
WPA_SUPPLICANT_VERSION := VER_0_8_X
BOARD_WPA_SUPPLICANT_DRIVER := NL80211
BOARD_HOSTAPD_DRIVER := NL80211
WIFI_FEATURE_HOSTAPD_11AX := true
PRODUCT_CFL_INCLUDE_PATHS += hardware/seekwave/wlan/wpa_supplicant_8_lib
include hardware/seekwave/wlan/configs/config-skw.mk
```

3 Driver 层

3.1 Kernel 内部编译

3.1.1 代码

- 将 skw6316 copy 到内核代码目录下的 drivers/net/wireless 目录下，并修改对应的 Kconfig 和 Makefile

Kconfig:

```
source "drivers/net/wireless/skw6316/Kconfig"
```

Makefile:

```
obj-$(CONFIG_WLAN_VENDOR_SEEKWAVE) += skw6316/
```

3.2 Kernel 外部编译

- 对比内部编译的主要差异是 driver 的默认配置项的不同，内部编译的时候会在 Kconfig 或者 kernel 的 defconfig 里面指定，外部编译的时候需要 Makefile 和 include/linux/platform_data/skw6316_config.h 里面指定；
- 编译的方法的差异；
 - 1) 修改 Makefile

```
ARCH ?= arm
KSRC ?= $(PWD)/../linux-4.91
CROSS_COMPILE ?= arm-linux-gnueabi-
```

根据实际需求修改对应的变量，或者通过 make 命令传递以上参数

- 2) 编译

已经修改 Makefile

```
make clean
```

```
make
```

未修改 Makefile

```
make KSRC=[kernel path] clean
```

```
make ARCH=[arch] CROSS_COMPILE=[cross_compile] KSRC=[kernel path]
```

4 配置和使用

4.1 log_level

- (1) 通过读取/proc/skwifid/log_level 获取当前的日志等级，

```
cat /proc/skwifid/log_level

Log Level: DEBUG    [ERROR|WARN|INFO|DEBUG|DETAIL]

command log: disable
event   log: disable
dump    log: disable
scan    log: disable
timer   log: disable
state   log: enable
work    log: disable
```

- (2) 修改日志等级

- echo INFO > /proc/skwifid/log_level
- 开启/关闭某个功能的 debug
echo [enable/disable]:\$func_name > /proc/skwifid/log_level
eg:
echo enable:event > /proc/skwifid/log_level
echo disable:scan > /proc/skwifid/log_level

4.2 Country code

- (1) EA6XXXX 无线驱动支持两种国家码方案；

方案一

该方案使用外部管制域数据库，即数据库由内核或用户层维护，国家码改变时，驱动处理内核通知完成国家码切换；

方案二

该方案驱动使用内部管制域数据库（打开 SKW_REGD_SELF_MANAGED 宏），此时驱动国家码不受全局国家码影响；可通过 ioctl 命令修改驱动当前国家码

- (2) 改国家码的方法

- hostapd.conf 里面指定 country_code=US
- 如果系统支持 debugfs 的时候，可以通过读写
/sys/kernel/debug/skwifi/chip1.usb/regdom 来获取和修改 country code
*:chip1.usb 根据 bus 类型变动，可以分别为 sub、sdio 或者 pcie
- Android 系统设置
- 设备根据周围 AP 的 country code 学习

4.3 skw config

在模块加载的时候驱动会根据编译的时候的宏配置和 skwifid.cfg 文件中来配置驱动的一个条件参数。

如果 skwifid.cfg 中设置了 overlay_mode=1, 驱动会根据 FW 提供的 chipid 信息读取对应的配置文件的内容, 用新的参数值覆盖前面的, eg: ssv6051-wifi.cfg。

配置文件内容如下:

[使用文档中的独特引言吸引读者的注意力, 或者使用此空间强调要点。要在此页面上的任何位置放置此文本框, 只需拖动它即可。]

4.4 MAC 地址

EA6XXXX 无线驱动通过以下方式获得 MAC 地址, 在无法获取 MAC 地址或 MAC 地址无效的情况下, 驱动生成随机 MAC 地址, 随机地址的格式为 FE:FD:FC:XX:XX:XX

1) EFUSE MAC 地址

芯片在产测中将 MAC 地址写入到 EFUSE 中, 驱动读取 EFUSE 地址作为默认 MAC 地址。

2) 加载驱动时传入 MAC 地址参数

加载驱动时, 通过参数传入驱动; 格式如下:

insmod skw6316.ko mac=0x00,0x11,0x22,0x33,0x44,0x55

3) NV 获得 MAC 地址

需要客户提供 NV 读写接口, 加载驱动时读取 NV 分区 MAC 地址作为默认 MAC 地址

Eg:

```
» if (skw_sync_cmd_event_version(wiphy) ||
»     skw_sync_chip_info(wiphy, &chip))
»     goto core_deinit;
»
» if (!is_valid_ether_addr(skw_mac))
»     rockchip_wifi_mac_addr(skw_mac);
»
» skw_setup_mac_address(wiphy, skw_mac, chip.mac);
```

MAC 地址优先顺序如下:

参数 MAC 地址 > NV MAC 地址 > EFUSE MAC 地址 > 随机 MAC 地址

4.5 固件校准文件

➤ EA6XXXX 芯片校准文件及固件保存在 firmware/目录下;

- 固件：
RAM_RW_KERNEL_DRAM.bin 和 ROM_EXEC_KERNEL_IRAM.bin
或者[芯片类型]_DRAM_[设备总线类型].bin、芯片类型]_IRAM_[设备总线类型].bin
eg: SWT6652_DRAM_USB.bin SWT6652_IRAM_USB.bin
- 校准文件
实际用的校准文件可以通过读取 core 信息获取, 参考查看软件信息章节
eg: Calib File: EA6316_SEEKWAVE_R04006.bin

4.6 查看软件信息

- 固件和校准文件信息

```
cat /proc/skwifid/chip1.usb/core

Timestamp: 53485987 - 53482.207650 (2024-08-20 09:54:27 UTC)

SKW Flags:      0x8300
TX Packets:     0
RX Packets:     0
txqlen_pending: 0
nr_lmac:        2
    credit[0]:  0 (inactive)
    credit[1]:  0 (inactive)

Calib File:      EA6316_SEEKWAVE_R04006.bin
FW Build:        20240823-02:36:09
FW Version:      rev31770-rev31783 (BSP-MAC)
BUS Type:        0x3c (usb)
Align Size:     1024
TX Limit:       15
```

- WiFi 驱动信息

```
cat /proc/skwifid/profile

Kernel Version:      5.10.157-ab440
Wi-Fi Driver:        2.0.240820.ca8fedd
Wi-Fi Branch:        skw6316_dev

CONFIG_SKW6316_STA_SME_EXT=y
CONFIG_SKW6316_SAP_SME_EXT=y
CONFIG_SKW6316_SCAN_RANDOM_MAC=y
CONFIG_SKW6316_LEGACY_P2P=y
CONFIG_SKW6316_TX_WORKQUEUE=y
CONFIG_SKW6316_HIGH_PRIORITY=y
CONFIG_SKW6316_VENDOR=y
CONFIG_SKW6316_REGD_SELF_MANAGED=y
# CONFIG_SKW6316_TDLS is not set
CONFIG_SKW6316_DFS_MASTER=y
# CONFIG_SKW6316_REPEATER_MODE is not set
CONFIG_SKW6316_EDMA=y
# CONFIG_SKW6316_OFFCHAN_TX is not set
CONFIG_SKW6316_CALIB_DPD=y
CONFIG_SKW6316_CALIB_BUS_SENSITIVE=y
CONFIG_SKW6316_6GHZ=y
# CONFIG_SKW6316_USB3_WORKAROUND is not set
# CONFIG_SKW6316_LOG_ERROR is not set
# CONFIG_SKW6316_LOG_WARN is not set
# CONFIG_SKW6316_LOG_INFO is not set
CONFIG_SKW6316_LOG_DEBUG=y
# CONFIG_SKW6316_LOG_DETAIL is not set
CONFIG_SKW6316_RX_REORDER_TIMEOUT=100
CONFIG_SKW6316_PROJECT_NAME="SEEKWAVE"
CONFIG_SKW6316_DEFAULT_COUNTRY=""
CONFIG_SKW6316_CHIP_ID="EA6316"
```

里面包含 kernel 的版本，驱动的版本和分支信息，同时包含编译驱动的配置信息。

5 FAQ

(1) Androi12 SAE 加密 24 小时固定断线

这个是 wpa_supplicant 的本身的 bug，在 Android13 的版本已经 fix，前期的版本需要合入 0001-SAE-Do-not-expire-the-current-PMKSA-cache-entry.patch

From b0f457b6191aa8ee329331ccbe0600326f7b1e2e Mon Sep 17 00:00:00 2001

From: Jouni Malinen <jouni@codeaurora.org>
Date: Mon, 18 Oct 2021 20:02:35 +0300
Subject: [PATCH] SAE: Do not expire the current PMKSA cache entry

There is no convenient mechanism for reauthenticating and generating a new PMK during an association with SAE. As such, forced PMK update would mean having to disassociate and reauthenticate which is not really desired especially when the default PMKLifetime is only 12 hours.

Postpone PMKSA cache entry expiration of the currently used entry with SAE until the association is lost. In addition, do not try to force the EAPOL state machine to perform reauthentication for SAE since that won't work.

Signed-off-by: Jouni Malinen jouni@codeaurora.org

- (2) WPS 关联过程中如果在同一时间同一信道有其它设备在做 WPS 会失败
Android10 及以前的版本有这个问题，Android11 以后是正常的，也是 wpa_supplicant 的问题。
- (3) 扫描特定 band
iwpriv wlan0 set/get bandcfg [0/1/2]
0:Auto 1:2G 2:5G
这个功能要在加载模块之后，就紧接着执行，不然会出现 FW 因为 offload scan 到之后会将其它 band 的上扫描的信息也上报。